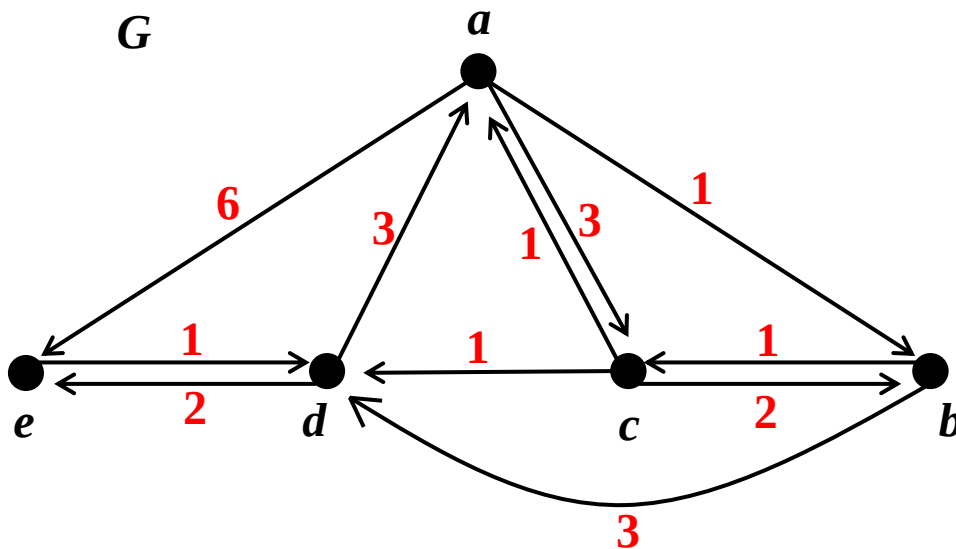


Algoritmo de Dijkstra

Problema: Dado um digrafo G com pesos positivos nas arestas, determinar **caminhos mínimos** de um vértice v a todos os demais vértices do digrafo.



matriz de pesos W

	a	b	c	d	e
a	0	1	3	∞	6
b	∞	0	1	3	∞
c	1	2	0	1	∞
d	3	∞	∞	0	2
e	∞	∞	∞	1	0

$$W(x, y) = \begin{cases} 0, & \text{se } x = y \\ \text{peso da aresta } xy, & \text{se houver } (x \neq y) \\ \infty, & \text{se não houver aresta } xy \ (x \neq y) \end{cases}$$

Algoritmo de Dijkstra

Realizando uma adaptação no algoritmo do exercício anterior, e fazendo com que a estrutura de dados D seja uma *fila de prioridades (heap)*, obtemos o **Algoritmo de Dijkstra**.

desmarcar todos os vértices; escolher uma raiz r ; $L(r) \leftarrow 0$; $\text{pai}(r) \leftarrow \text{null}$; inserir r em D
para todo $w \neq r$ faça : $L(w) \leftarrow \infty$; $\text{pai}(w) \leftarrow \text{null}$; inserir w em D com prioridade $L(w)$
enquanto $D \neq \emptyset$ faça

seja v o primeiro elemento de D -- v é elemento de menor valor $L(v)$

remover v de D ; marcar v

para todo vértice w em $N_{\text{out}}(v)$ desmarcado faça

se $L(v) + W(v, w) < L(w)$ então

$L(w) \leftarrow L(v) + W(v, w)$

reposicionar w em D (de acordo com sua nova prioridade)

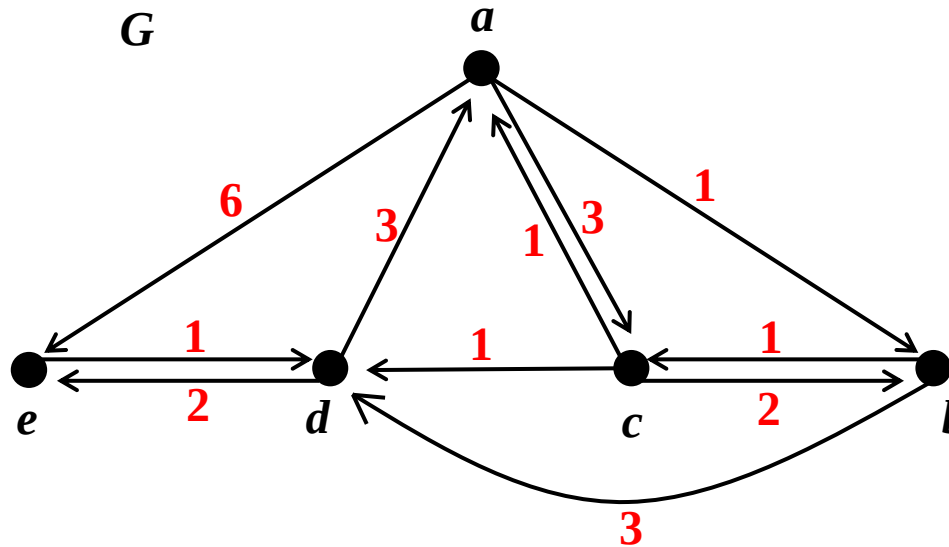
$\text{pai}(w) \leftarrow v$

fim-para

fim-enquanto

Algoritmo de Dijkstra

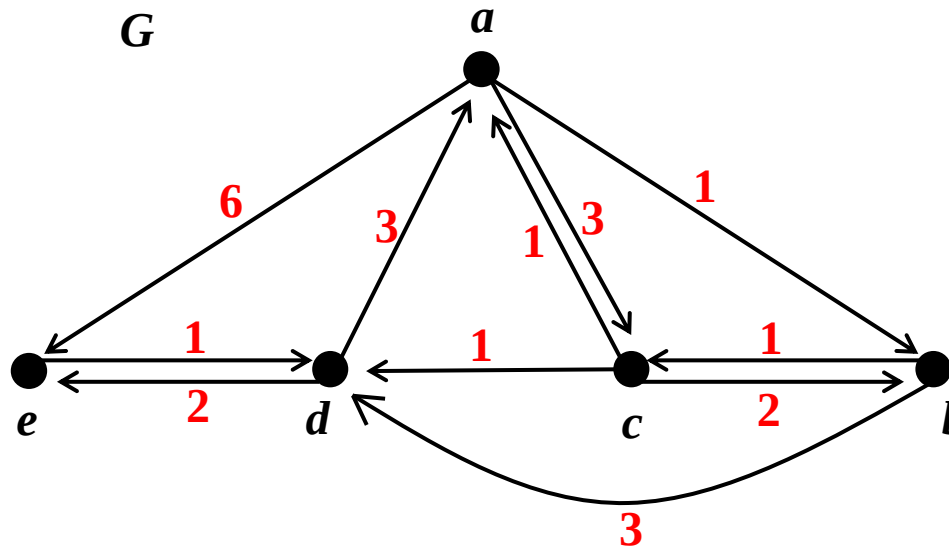
raiz: a



<i>v</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>
<i>L(v)</i>	0	∞	∞	∞	∞
<i>pai(v)</i>	<i>null</i>	<i>null</i>	<i>null</i>	<i>null</i>	<i>null</i>

Algoritmo de Dijkstra

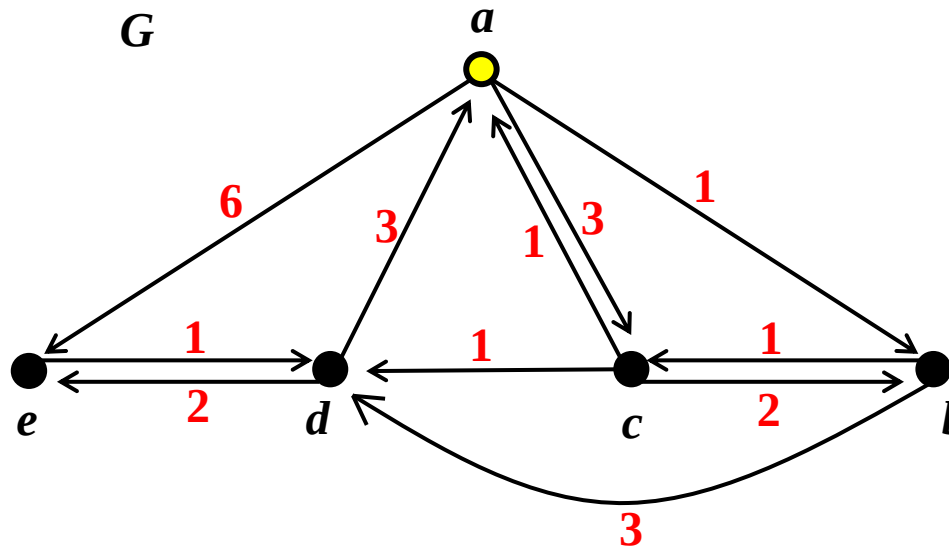
raiz: a



<i>v</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>
<i>L(v)</i>	0	∞	∞	∞	∞
<i>pai(v)</i>	<i>null</i>	<i>null</i>	<i>null</i>	<i>null</i>	<i>null</i>

Algoritmo de Dijkstra

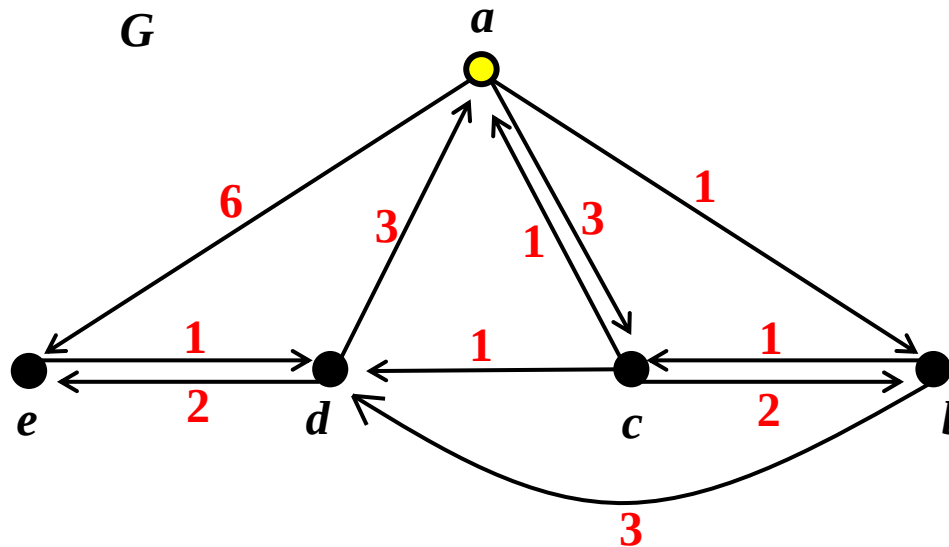
raiz: a



<i>v</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>
<i>L(v)</i>	0	∞	∞	∞	∞
<i>pai(v)</i>	<i>null</i>	<i>null</i>	<i>null</i>	<i>null</i>	<i>null</i>

Algoritmo de Dijkstra

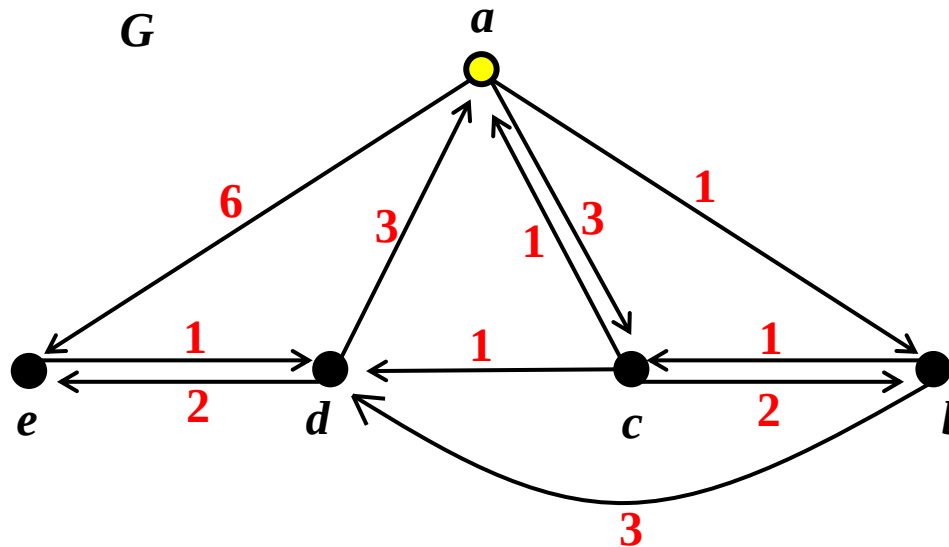
raiz: a



v	a	b	c	d	e
$L(v)$	0	1	∞	∞	∞
$pai(v)$	<i>null</i>	a	<i>null</i>	<i>null</i>	<i>null</i>

Algoritmo de Dijkstra

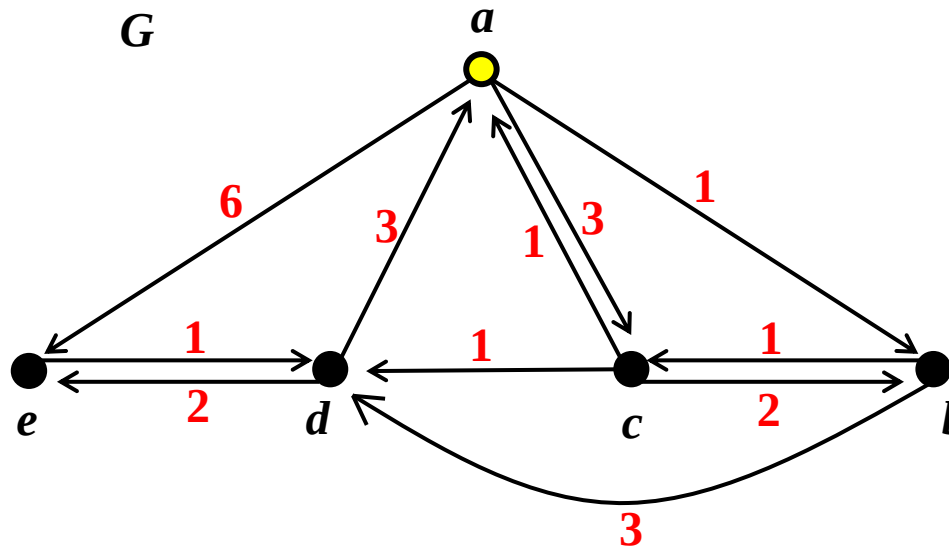
raiz: a



<i>v</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>
<i>L(v)</i>	0	1	3	∞	∞
<i>pai(v)</i>	<i>null</i>	<i>a</i>	<i>a</i>	<i>null</i>	<i>null</i>

Algoritmo de Dijkstra

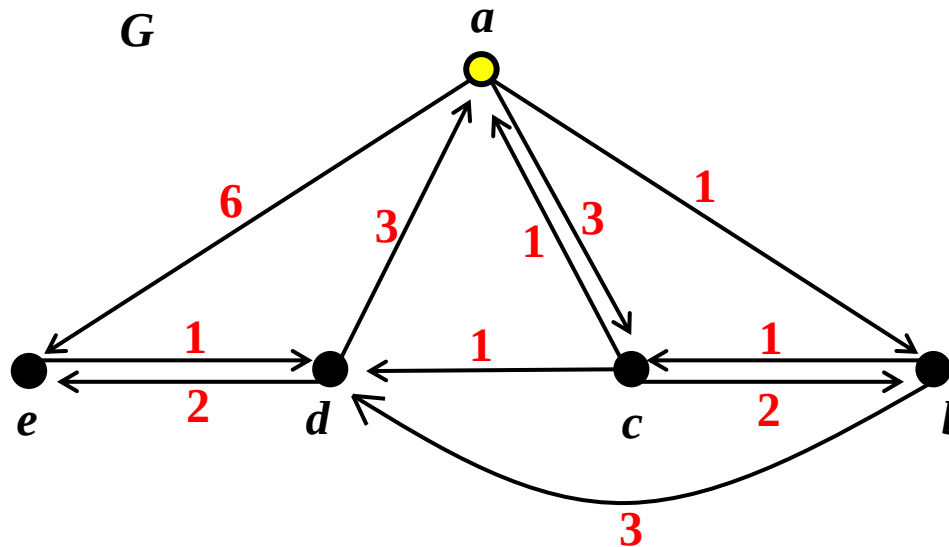
raiz: a



<i>v</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>
<i>L(v)</i>	0	1	3	∞	6
<i>pai(v)</i>	<i>null</i>	<i>a</i>	<i>a</i>	<i>null</i>	<i>a</i>

Algoritmo de Dijkstra

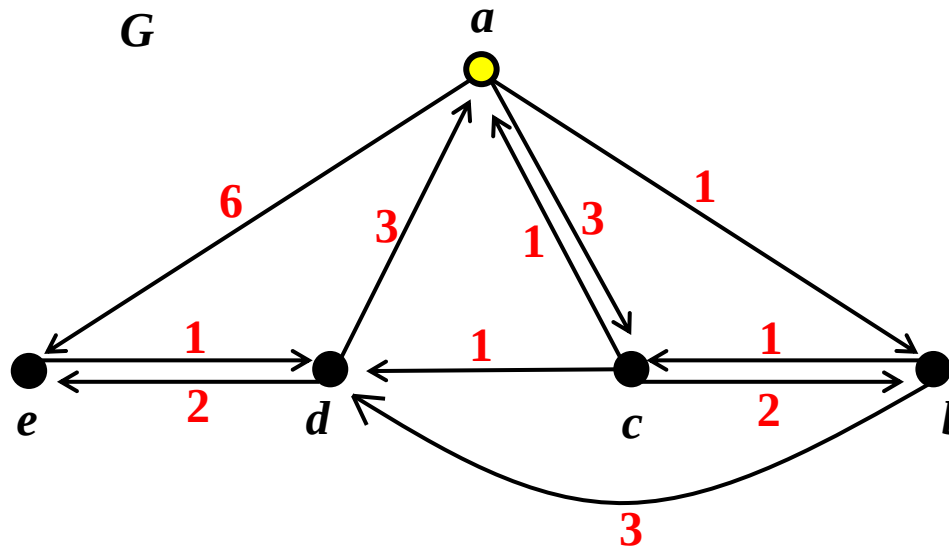
raiz: a



<i>v</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>
<i>L(v)</i>	0	1	3	∞	6
<i>pai(v)</i>	<i>null</i>	<i>a</i>	<i>a</i>	<i>null</i>	<i>a</i>

Algoritmo de Dijkstra

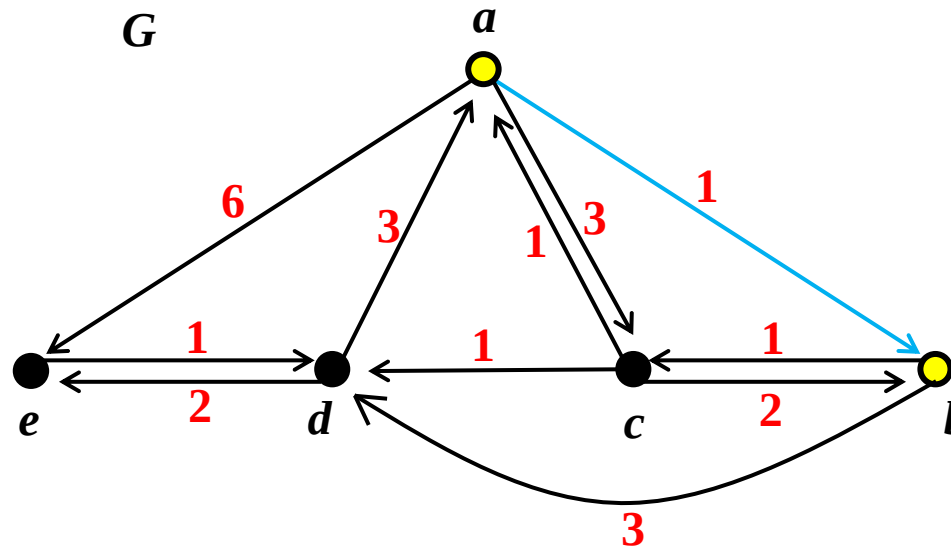
raiz: a



<i>v</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>
<i>L(v)</i>	0	1	3	∞	6
<i>pai(v)</i>	<i>null</i>	<i>a</i>	<i>a</i>	<i>null</i>	<i>a</i>

Algoritmo de Dijkstra

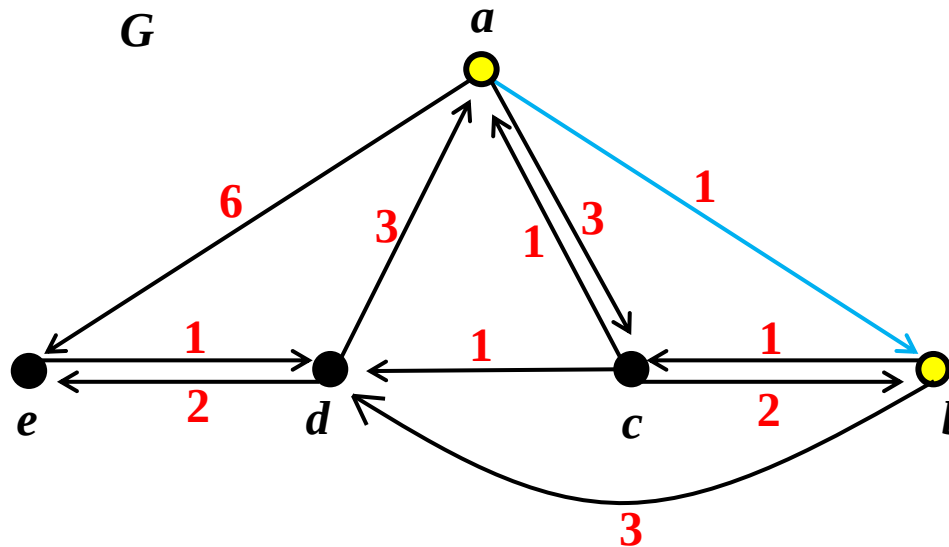
raiz: a



v	a	b	c	d	e
$L(v)$	0	1	3	∞	6
$pai(v)$	$null$	a	a	$null$	a

Algoritmo de Dijkstra

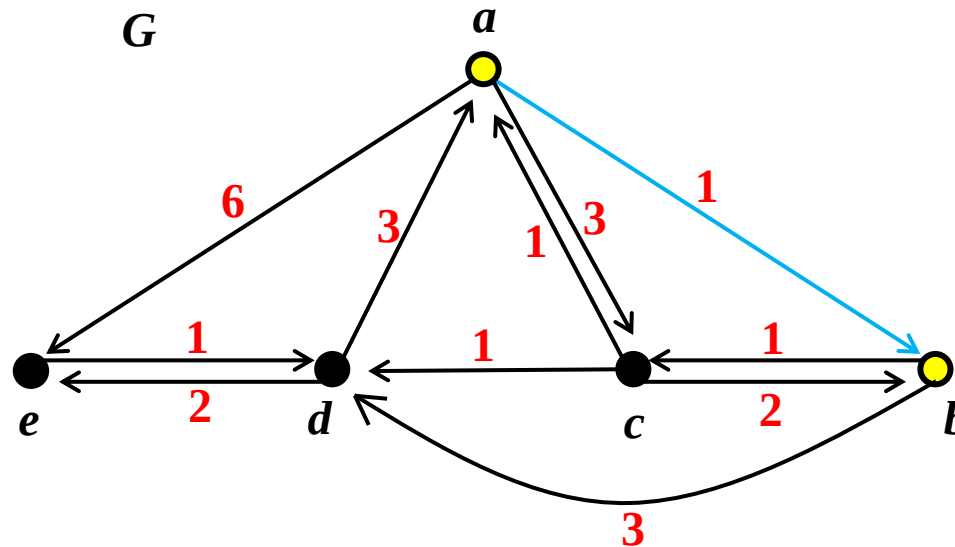
raiz: a



<i>v</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>
<i>L(v)</i>	0	1	2	∞	6
<i>pai(v)</i>	<i>null</i>	<i>a</i>	<i>b</i>	<i>null</i>	<i>a</i>

Algoritmo de Dijkstra

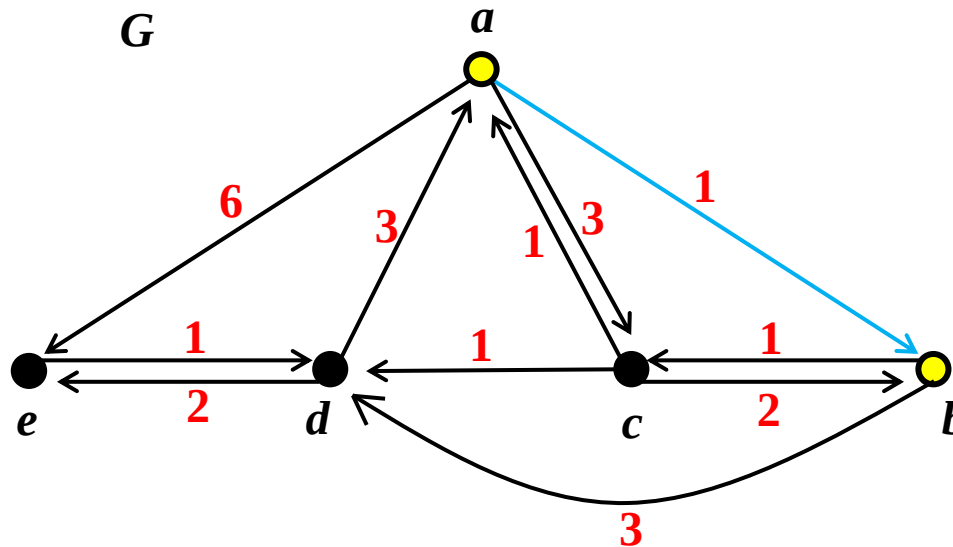
raiz: a



v	a	b	c	d	e
$L(v)$	0	1	2	4	6
$pai(v)$	<i>null</i>	a	b	b	a

Algoritmo de Dijkstra

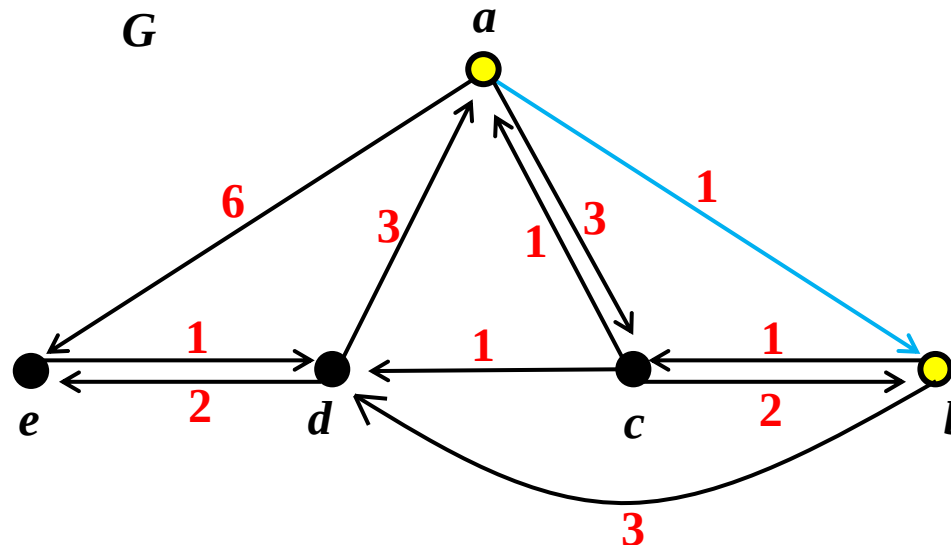
raiz: a



<i>v</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>
<i>L(v)</i>	0	1	2	4	6
<i>pai(v)</i>	<i>null</i>	<i>a</i>	<i>b</i>	<i>b</i>	<i>a</i>

Algoritmo de Dijkstra

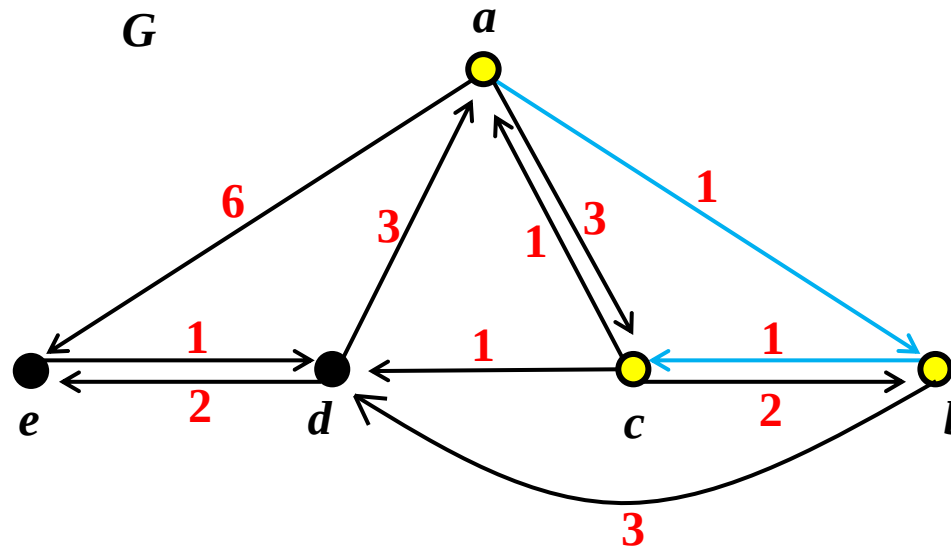
raiz: a



<i>v</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>
<i>L(v)</i>	0	1	2	4	6
<i>pai(v)</i>	<i>null</i>	<i>a</i>	<i>b</i>	<i>b</i>	<i>a</i>

Algoritmo de Dijkstra

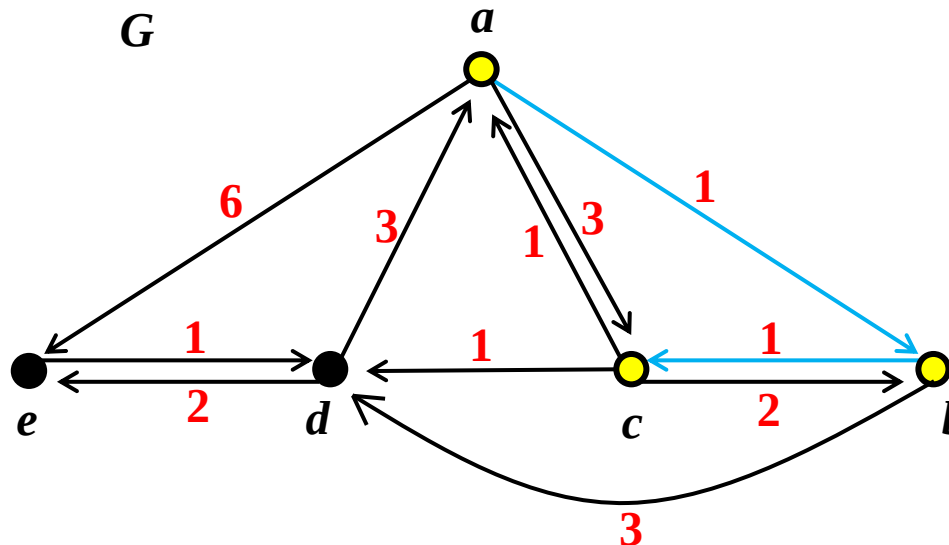
raiz: a



<i>v</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>
<i>L(v)</i>	0	1	2	4	6
<i>pai(v)</i>	<i>null</i>	<i>a</i>	<i>b</i>	<i>b</i>	<i>a</i>

Algoritmo de Dijkstra

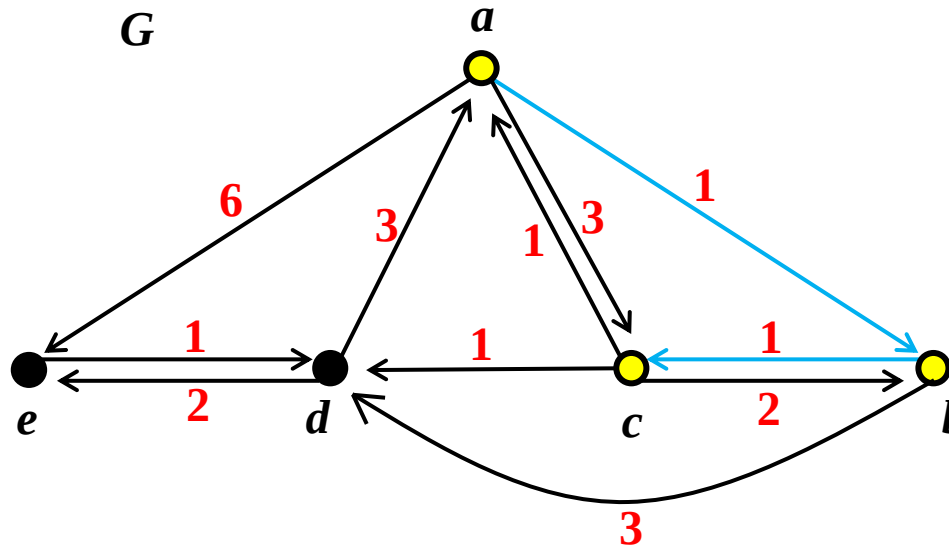
raiz: a



<i>v</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>
<i>L(v)</i>	0	1	2	3	6
<i>pai(v)</i>	<i>null</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>a</i>

Algoritmo de Dijkstra

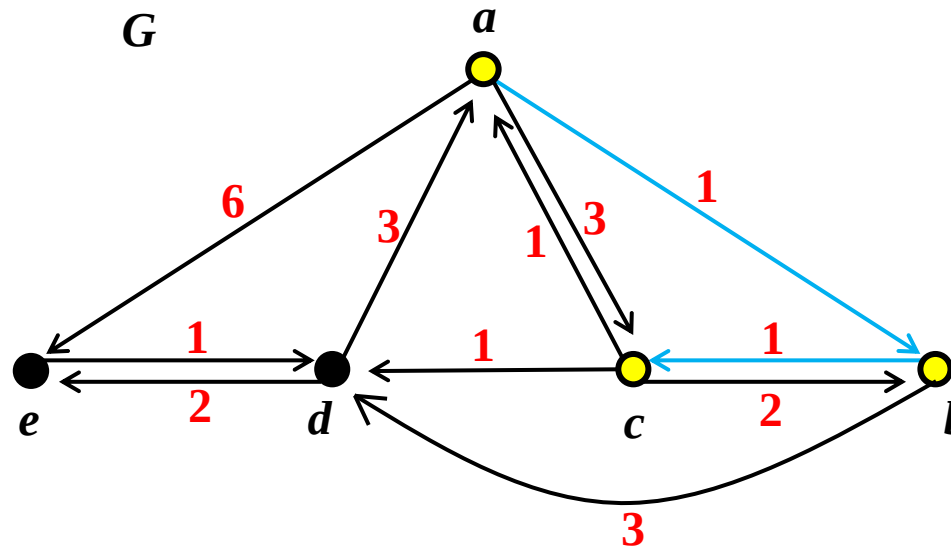
raiz: a



<i>v</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>
<i>L(v)</i>	0	1	2	3	6
<i>pai(v)</i>	<i>null</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>a</i>

Algoritmo de Dijkstra

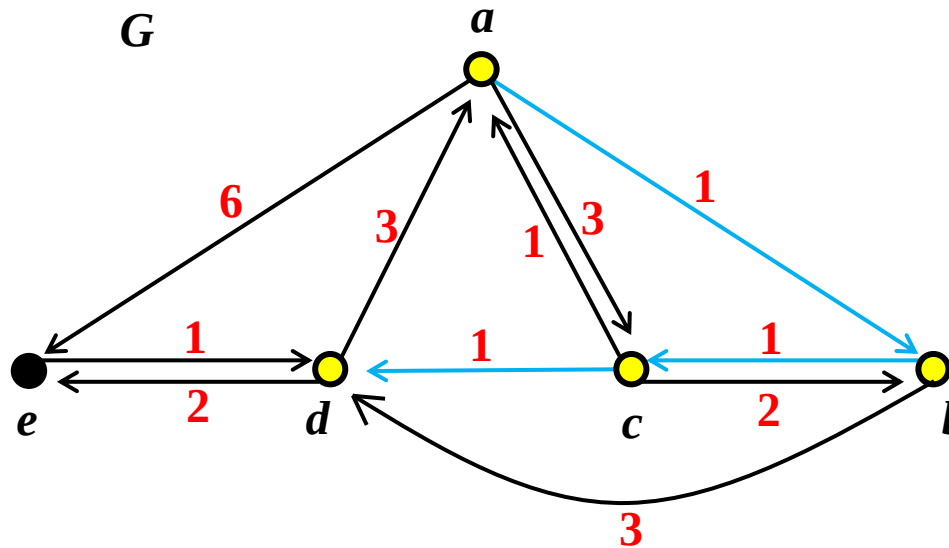
raiz: a



<i>v</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>
<i>L(v)</i>	0	1	2	3	6
<i>pai(v)</i>	<i>null</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>a</i>

Algoritmo de Dijkstra

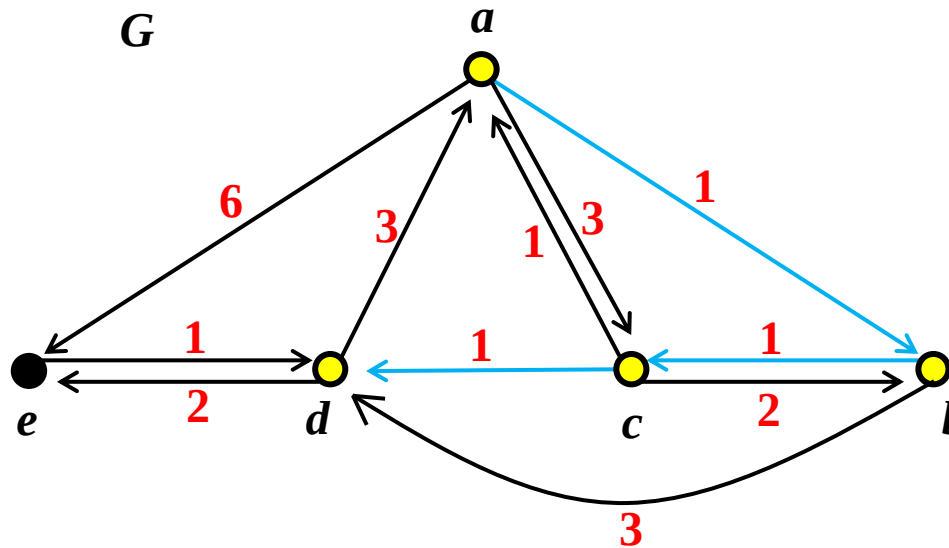
raiz: a



v	a	b	c	d	e
$L(v)$	0	1	2	3	6
$pai(v)$	$null$	a	b	c	a

Algoritmo de Dijkstra

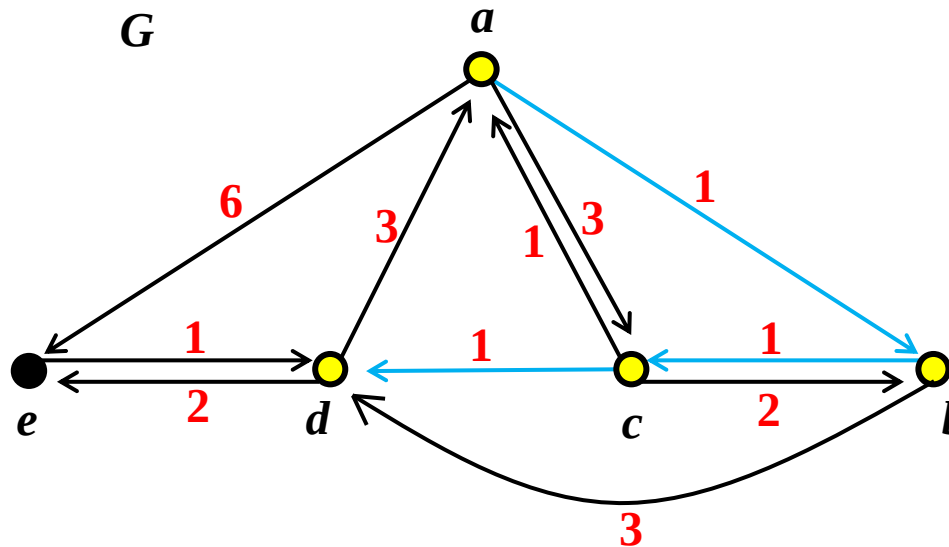
raiz: a



<i>v</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>
<i>L(v)</i>	0	1	2	3	5
<i>pai(v)</i>	<i>null</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>

Algoritmo de Dijkstra

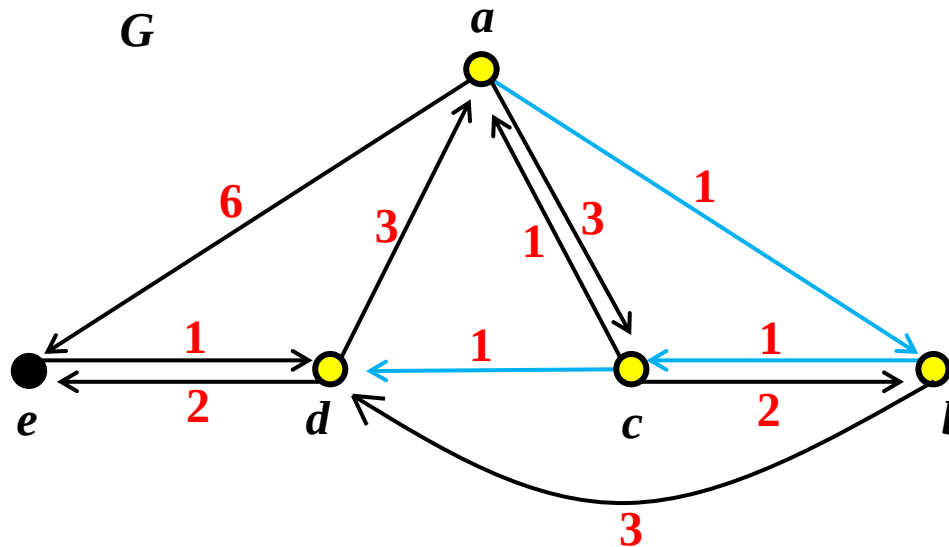
raiz: a



<i>v</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>
<i>L(v)</i>	0	1	2	3	5
<i>pai(v)</i>	<i>null</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>

Algoritmo de Dijkstra

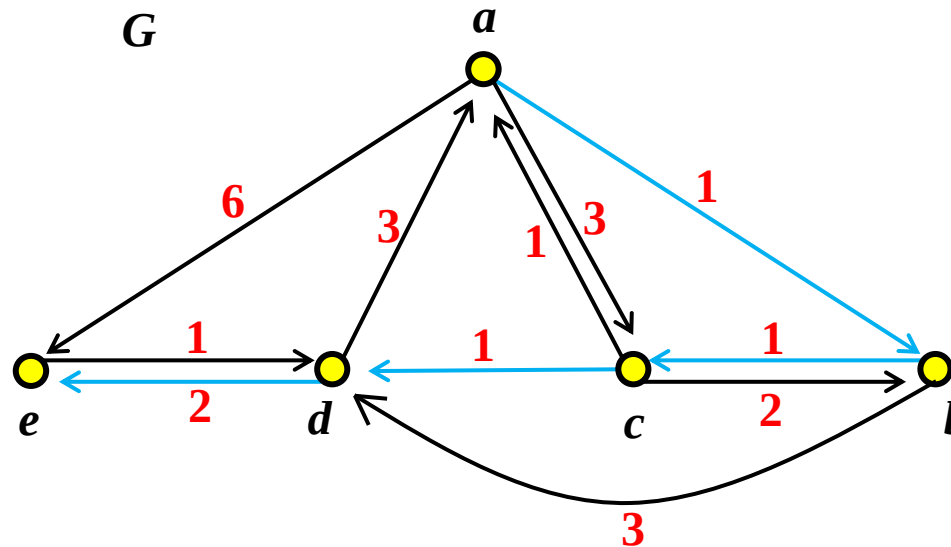
raiz: a



<i>v</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>
<i>L(v)</i>	0	1	2	3	5
<i>pai(v)</i>	<i>null</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>

Algoritmo de Dijkstra

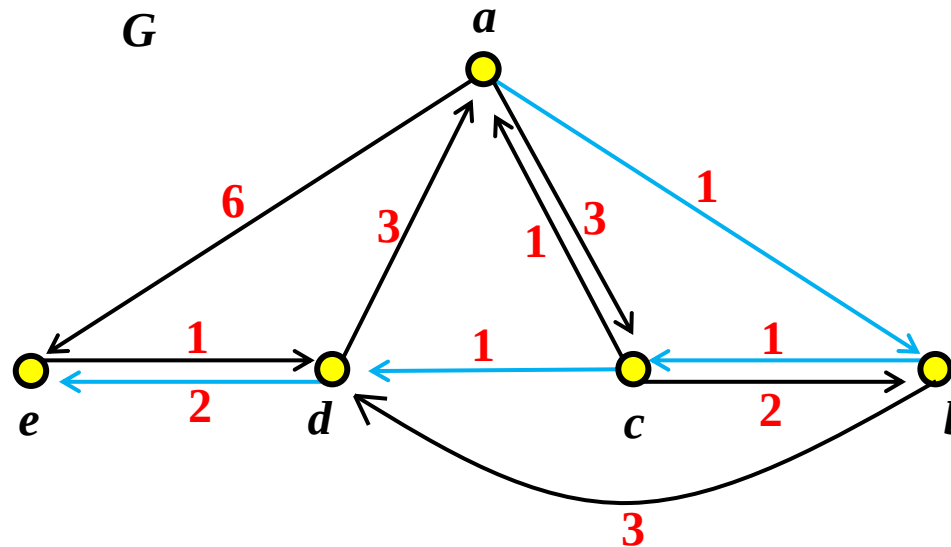
raiz: a



<i>v</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>
<i>L(v)</i>	0	1	2	3	5
<i>pai(v)</i>	<i>null</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>

Algoritmo de Dijkstra

raiz: a



<i>v</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>
<i>L(v)</i>	0	1	2	3	5
<i>pai(v)</i>	<i>null</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>

FIM!

Algoritmo de Dijkstra

Complexidade

A complexidade do Algoritmo de Dijkstra é dominada pelas seguintes operações:

- n remoções da estrutura D -- $O(n \log n)$
- n inserções na estrutura D -- $O(n \log n)$
- no máximo m atualizações de prioridade (rótulos $L(v)$) na estrutura D -- $O(m \log n)$
- **TOTAL: $O((n + m) \log n)$**
- Grafo esparso ($m = O(n)$): **$n \log n$**
- Grafo denso ($m = O(n^2)$): **$n^2 \log n$**
- Rodando n vezes para todas as possíveis raízes:
 $n^2 \log n + nm \log n$

Algoritmo de Dijkstra

Questão: Como determinar os caminhos mínimos?

Solução: Observe que os ponteiros $pai(w)$ determinam naturalmente uma árvore T , chamada de **árvore de caminhos mínimos** (a partir da raiz escolhida). Na simulação, esta árvore T está formada pelas arestas azuis.

Para determinar um caminho mínimo da raiz v até um vértice w , basta tomar o **caminho de v a w em T** . O custo deste caminho é precisamente $L(w)$.

Algoritmo de Dijkstra

Questão: Como achar os caminhos mínimos da raiz a todos os demais vértices em um grafo G **não** direcionado?

Solução: Basta transformar G em um digrafo H da seguinte forma: fazer $V(H) = V(G)$ e, para cada aresta não direcionada xy com peso p em G , criar em H duas arestas direcionadas **xy** e **yx** , ambas com peso p . Por outro lado, se não existe aresta xy em G , criar em H duas arestas direcionadas **xy** e **yx** com peso ∞ . Basta então aplicar o algoritmo de Dijkstra a H .

Algoritmo de Dijkstra

Exercício: Mostre que o Algoritmo de Dijkstra se comporta como uma busca em largura se os pesos de todas as arestas são iguais a um. Neste caso, a árvore de caminhos mínimos é uma árvore de largura, e cada rótulo $L(w)$ é igual à distância (em arestas) da raiz até w .